

FS Assignment-1

Subject Name: Full Stack Web Development

Name: Ramayanam Satya Venkata Ramana Prasad

Roll No: A24126552266

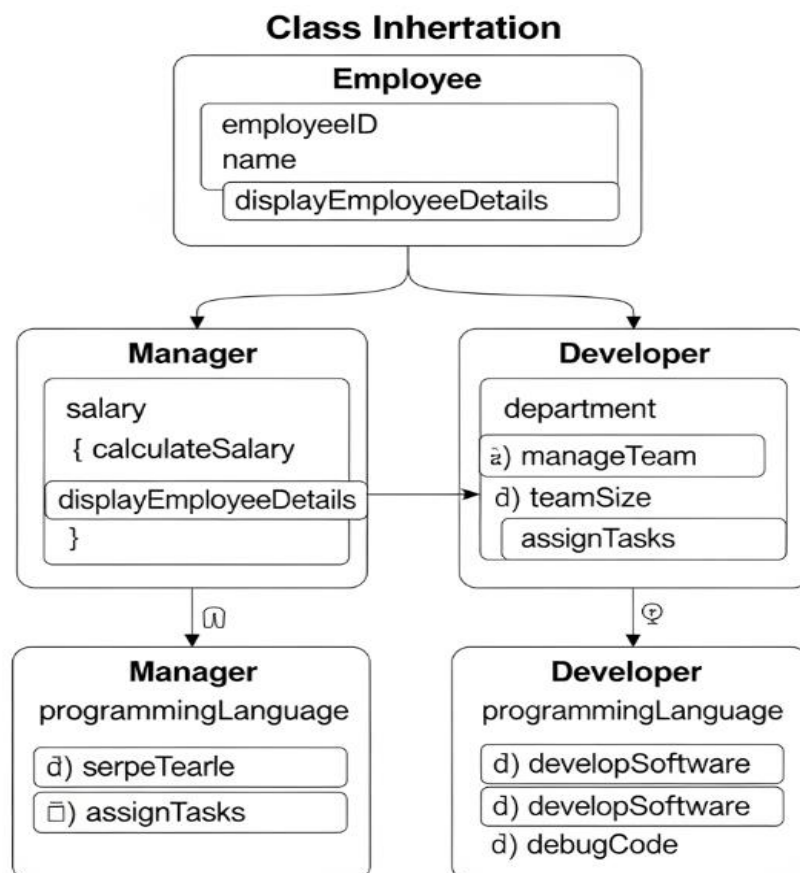
Branch & Section: CSM-B

Date Submitted: 27/8/26

*INHERITANCE:

There are **5 classical types of inheritance** in OOPs, and JavaScript can express all of them despite being prototype-based rather than class-based. Here's a quick overview with JS examples for each they are:

1. Single Inheritance
2. Multi-level Inheritance
3. Hierarchical Inheritance
4. Multiple Inheritance
5. Hybrid Inheritance



1.Single Inheritance: Structure: $A \rightarrow B$

A single class (child/derived class) inherits properties and methods from **exactly one** parent (base/super) class.

```
//=====SINGLE INHERITANCE=====//
class Satya{
  constructor(name){
    this.name=name;
  }
  talk(){
    console.log("hello my name is Satya!");
  }
}

class Sravan extends Satya{
  friendTalk(){
    console.log("hello Satya i am your friend");
  }
}
```

2.Multi-level Inheritance: **Structure:** $A \rightarrow B \rightarrow C$

A class inherits from a derived class, which itself inherited from another class — forming a **chain** of inheritance across multiple levels.

```
//=====MULTI-LEVEL INHERITANCE=====//
class Srikar extends Satya{
  friendlaughes(){
    console.log("hello satya i am your friend and i am your motivation");
  }
}
```

3. Hierarchical Inheritance: **Structure:** $A \rightarrow B, A \rightarrow C, A \rightarrow D$

Multiple child classes inherit from a **single parent** class, independently of each other.

```
//=====HIERARCHICAL INHERITANCE=====//
class Abhishek extends Satya{
  friendsmiles(){
    console.log("hello satya i am your friend that supports you always");
  }
}
```

4. Multiple Inheritance: **Structure:** A, B → C

A single class inherits properties and behaviors from more than one parent class simultaneously.

```
//=====MULTIPLE INHERITANCE=====//
const Canread = Base => class extends Base{
  read(){
    console.log("i am reading");
  }
};

const Canwrite = Base => class extends Base{
  write(){
    console.log("i am writing");
  }
};

const Canshow = Base => class extends Base{
  show(){
    console.log("i am showing my skills");
  }
};

class Harsha extends Canread(Canwrite(Canshow(Satya))){
  expertSkills(){
    console.log("i am an expert");
  }
}
```

5. Hybrid Inheritance: Structure: Mixed — e.g., hierarchical + multilevel, or hierarchical + multiple

A combination of two or more of the above inheritance types used together in the same class hierarchy.

```
//=====HYBRID INHERITANCE=====//
console.log("-----Single-----");
const sr = new Sravan("Somya");
sr.talk();
sr.friendTalk();

console.log("-----Multi-Level-----");
const sk = new Srikar("Dharani");
sk.talk();
sk.friendlaughs();

console.log("-----Hierarchical-----");
const ab = new Abhishek("Srinivas");
ab.talk();
ab.friendsmiles();

console.log("-----Multiple-----");
const hr = new Harsha("Surya");
hr.talk();

console.log("-----Hybrid-----");
const hy = new Harsha("Abhishek");
hy.talk();
```

Output:

```
S> node Assignment-1.js
-----Single-----
hello my name is Satya!
hello Satya i am your friend
-----Multi-Level-----
hello my name is Satya!
hello satya i am your friend and i am your motivation
-----Hierarchical-----
hello my name is Satya!
hello satya i am your friend that supports you always
-----Multiple-----
hello my name is Satya!
-----Hybrid-----
hello my name is Satya!
```

Result:

Shown every inheritance type in Javascript .

Signature of Faculty:_____